Name- Prince Verma

Roll no.- 2401010253

Course- B.tech CSE Core (Section C)

Subject- Analysis and design of Algorithms

Lab Assignment 2

Task 1: Divide and Conquer Algorithm

```
import time import
random


def merge_sort(arr):
   if len(arr) <= 1:        return
arr     mid = len(arr)//2     left =
merge_sort(arr[:mid])     right =
merge_sort(arr[mid:])     return
merge(left, right)


def merge(left, right):
   result = []     i = j = 0     while i <
len(left) and j < len(right):        if
left[i] < right[j]:
          result.append(left[i])
i+=1        else:
          result.append(right[j])
j+=1     result.extend(left[i:])
result.extend(right[j:])     return
result
```

```
arr = [random.randint(1,1000) for _ in range(100)]

start = time.time() merge_sort(arr) end =

time.time()


print("Merge Sort Time:", end-start)
```

Output:

```
Merge Sort Time: 0.0007667541503906 25
```

Task 2: Sorting Performance Comparison

```
import time import

random


def bubble_sort(arr):     n = len(arr)

for i in range(n):        for j in range(0,

n-i-1):            if arr[j] > arr[j+1]:

arr[j], arr[j+1] = arr[j+1], arr[j]

sizes = [100,200,300]


for size in sizes:

    arr = [random.randint(1,1000) for _ in range(size)]

start = time.time()     merge_sort(arr.copy())

merge_time = time.time() - start



    start = time.time()

bubble_sort(arr.copy())     bubble_time

= time.time() - start
```

```
    print("Size:", size)
print("Merge:", merge_time)
print("Bubble:", bubble_time)
```

Output:

```
Size: 100
Merge: 0.00022959709167480 47
Bubble: 0.00041866302490234375
Size: 200
Merge: 0.0003790855407714844
Bubble: 0.0016236305236816406
Size: 300
Merge: 0.0005674362182617188
Bubble: 0.0036232471466064453
```

Task 3: Greedy Algorithm (Fractional Knapsack)

```
def fractional_knapsack(weights, values, capacity):
    ratio = sorted(
        [(values[i]/weights[i], weights[i], values[i])
for i in range(len(weights))], reverse=True)     total
= 0

    for r, w, v in ratio:
if capacity >= w:
capacity -= w
total += v        else:
            total += r * capacity
            break
return total

weights = [10,20,30] values
= [60,100,120] capacity =
50
```

```
print("Maximum Profit:", fractional_knapsack(weights, values, capacity))
```

Output:

```
Maximum Profit: 240.0
```

Task 4: Dynamic Programming (0/1 Knapsack)

```
def knapsack(W, wt, val, n):
    dp = [[0 for x in range(W + 1)] for x in range(n + 1)]


    for i in range(n + 1):
        for w in range(W + 1):
if i == 0 or w == 0:
            dp[i][w] = 0
elif wt[i-1] <= w:
            dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]],
                           dp[i-1][w])
else:
            dp[i][w] = dp[i-1][w]


    return dp[n][W] val
= [60, 100, 120] wt =
[10, 20, 30]

W = 50
n = len(val)


print("Maximum Value:", knapsack(W, wt, val, n))
```

Output:

```
Maximum Value: 220
```

Task 5: Dijkstra Algorithm

```
import sys from itertools import
permutations def tsp(graph):     n =
len(graph)     vertices =
list(range(n))

    min_path = sys.maxsize     for perm in
permutations(vertices[1:]):
        cost = 0
k = 0

        for j in perm:
            cost += graph[k][j]
            k = j

        cost += graph[k][0]
min_path = min(min_path, cost)

    return min_path

graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]
```

print("Minimum TSP cost:", tsp(graph)) **Output:**

```
Minimum TSP cost: 80
```

Github; https://github.com/princeverma2052005-sys